



Universidade Federal do Rio Grande do Norte

Instituto Metr pole Digital

IMD3005 - MLOPS

Discente: Maria Clara Silva Brito

Relatório do laboratório 1

1. Dados

Para realizar o pipeline desse Laboratório, foram utilizados os dados a API OpenF1, uma plataforma de código aberto que fornece dados de Fórmula1 em tempo real e históricos, fornecendo diversos atributos, relacionados a tempos de volta, telemetria do carro, informações do piloto, mensagens da direção de prova entre outros. Escolhi essa API para ser utilizada pois a corrida é algo que eu gosto e preenchia os requisitos pedidos na atividade.

2. Feature Engineering

Os dados desta API possuem diversas colunas com informações que não são estritamente necessárias ou que podem ser combinadas e transformadas em novos atributos. A cada volta completada por um piloto, eram fornecidas informações detalhadas sobre o tempo de conclusão e a velocidade do carro. Estes dados eram extraídos em diversos setores e pontos específicos da pista, proporcionando uma visão mais ampla do que apenas o cruzamento da linha de chegada.

Para lidar com essa complexidade e preparar a base para modelagem, a etapa de transformação do pipeline refinou o conjunto de dados aplicando uma série de técnicas integradas. Inicialmente, foi realizada a seleção de features com a manutenção estrita das colunas de interesse analítico (`driver_number`, `lap_number`, `lap_duration` e `is_pit_out_lap`). Isso permitiu o descarte de variáveis de telemetria espacial bruta, ou seja, algumas informações brutas, que eram extraídas dos sensores, como posição física do carro no espaço, que forneciam milhares de coordenadas X e Y, isso já reduzia o custo computacional.

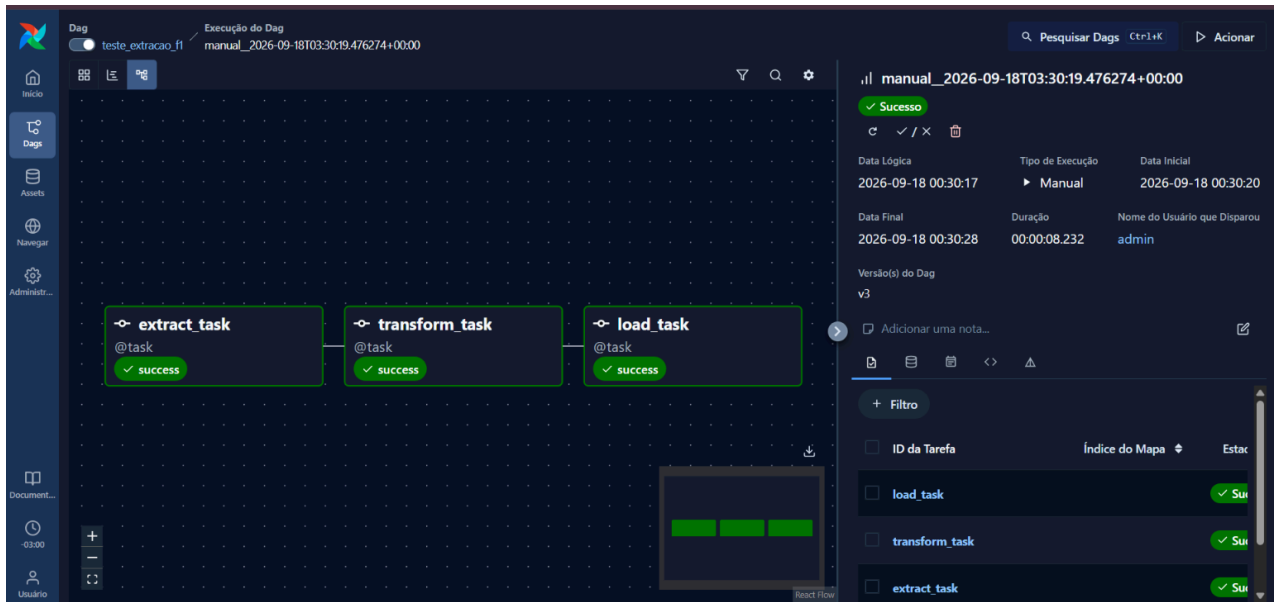
Em seguida, foi criada a variável `prev_lap_duration`, que armazena o tempo da volta anterior do piloto. Essa técnica, conhecida como lag temporal, é muito importante porque permite analisar o ritmo da corrida. Ao colocar o tempo anterior ao lado do tempo atual, os modelos de previsão conseguem entender se o piloto está ganhando ou perdendo velocidade ao longo do tempo, em vez de analisar cada volta de forma isolada, o que acontecia antes e a API guardava cada informação dessa separadamente.

Também, para garantir que todas as informações pudessem ser processadas matematicamente por modelos de Machine Learning, executou-se a codificação categórica da variável `is_pit_out_lap`, transformando-a de seu formato booleano original para uma representação binária inteira (1 e 0).

Por fim, o pipeline realizou o tratamento de omissões com a remoção de todos os valores nulos do DataFrame, limpando tanto eventuais falhas de captação da própria API quanto as linhas vazias geradas intencionalmente pela técnica de lag na primeira volta, garantindo assim a integridade estrutural do arquivo gerado.

3. Pipeline no Airflow

Atendendo aos requisitos da atividade, foi montado um pipeline com um DAG que continha as três tarefas essenciais da arquitetura ETL.



A tarefa de extração realizava a requisição HTTP à API e trazia para a memória os dados brutos em formato JSON. Na etapa de transformação, ocorreu toda a parte de engenharia de features: foram criadas as variáveis temporais e os dados nulos foram removidos, deixando o conjunto de dados o mais adequado possível. Por fim, a etapa de carga recebia o dicionário com os dados transformados, convertia os registros em um DataFrame utilizando a biblioteca Pandas e salvava o resultado em um arquivo .csv.

O DAG foi executado manualmente através do botão "Trigger DAG" na interface web do Airflow. A verificação do resultado ocorreu em duas etapas: visualmente, ao confirmar que todos os nós do DAG ficaram verdes na interface gráfica, e fisicamente, validando a criação e o conteúdo do arquivo features.csv no diretório do sistema operacional (WSL).

Para projetos de Machine Learning, o Apache Airflow torna-se fundamental devido à sua robusta capacidade de orquestração. Ele garante, por exemplo, que um modelo só comece a ser treinado após a transformação dos dados estar totalmente concluída. Além disso, a plataforma centraliza o estado de todas as execuções e gera logs isolados por tarefa, o que permite identificar rapidamente onde ocorreu uma falha. Outro grande benefício é a sua escalabilidade, pois suporta processamento paralelo e pode ser distribuído em múltiplos nós conforme o volume de dados cresce.

Na engenharia de dados, a escolha entre ETL e ELT depende muito da infraestrutura e do volume de dados que estamos lidando. O ETL primeiro extrai os dados, realiza as transformações em um servidor intermediário e só depois carrega os dados limpos no destino final, dessa forma, ele se torna apropriado quando o destino é simples, os dados cabem na memória e há necessidade de bibliotecas complexas em Python.

Já o ELT inverte a ordem, extrai e carrega os dados brutos diretamente para um Data Warehouse. A transformação ocorre dentro do próprio banco de dados de destino, utilizando SQL. É a abordagem padrão

hoje para projetos grandes, pois aproveita o enorme poder de processamento distribuído da nuvem para transformar uma gigantesca quantidade de dados sem gargalos de memória em servidores intermediários.

Durante a realização deste laboratório, a principal dificuldade encontrada esteve relacionada à instalação e execução do Airflow no sistema operacional Windows por meio do WS. Houve um desafio técnico ao tentar sincronizar o arquivo da DAG criado localmente com a pasta reconhecida pelo Airflow. Com o auxílio de ferramentas de Inteligência Artificial para depuração do ambiente, foi possível identificar e corrigir o mapeamento do diretório. Após esse ajuste, a DAG foi reconhecida corretamente pela interface, permitindo a execução das tarefas com sucesso